A fully client-side C++ IDE for CS1 using WebAssembly

Joshua Delgadillo
Stanford University
Palo Alto, CA
joshdelg@stanford.edu

Chris Piech
Stanford University
Palo Alto, CA
piech@cs.stanford.edu

Abstract

In this paper, we present a novel integrated development environment (IDE) for compiling, linking, executing, and debugging C++ entirely in the browser, without depending on an external server. The system is targeted at introductory CS courses, providing several useful features such as a no-setup environment, line-by-line debugging, support for no-cost shareable snippets, offline access, and usability on tablets/mobile phones, and a severless architecture with negligible cost at scale.

Our toolchain relies on WebAssembly (WASM), a portable bytecode designed as a cross-platform compilation target that can be run via a VM in any environment, including a browser. We combine several existing WASM ecosystem tools– the WASI Syscall specification, a Clang compiled to WASM, a WASI Syscall implementation– as well as leverage our own custom syscalls and an open-source WASM interpreter to build a completely integrated environment.

When compared against existing in-the-browser solutions, our IDE offers the union of their features while losing only a second of compile/execution time compared to the median tool, all while avoiding dependence on a server.

We hope to provide our tool as an open-source, accessible, low-cost solution for intro computer science courses seeking to teach important fundamentals in C++.

ACM Reference Format:
Joshua Delgadillo and Chris Piech. 2025. A fully client-side C++ IDE for CS1 using WebAssembly. In *Proceedings of (CS191W Winter '25).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

In the past few years, Massive, Open, Online Courses (MOOCs) have emerged to teach introductory computer science skills [10] [21]. These courses were notable in their accessibility and reach. Due to their online nature, many have opted for in-browser Integrated Development Environments (IDEs) to provide a completely integrated content and programming platform. Code in Place [19], one such platform based out of Stanford, currently utilizes an in-browser Python IDE for its course [8]. This IDE is a entirely client-side, meaning it does not require an additional server to execute code, saving thousands each month at Code in Place's scale. However, Code in Place's content has been limited to Python as constrained by the IDE's lack of support of other languages. So, in preparation for the next Code in Place iteration, we built a fully-client side C++ IDE capable of compiling, linking, executing, and debugging student code in-browser, without an external server, and interacting with the platform's error handling, terminal, and graphics functionality.

1.1 Benefits of an In-Browser IDE

We define an **in-browser IDE** as any IDE that is accessible through a web-browser, whether code execution takes place in the browser or via an external server accessed through HTTP requests or WebSockets [16]. A specifically **client-side IDE** is an in-browser IDE that has *no dependence* on an external server for code execution and does so in the browser via compiling to WebAssembly (WASM) [25], transpiling to Javascript, or interpreting.

In-browser IDEs on their own provide a host of benefits. They are OS-agnotistc, providing a unified development environment for each student, something especially critical in massive classrooms. This is even more relevant for C++ tooling, which frequently requires more complex setup (i.e. OS-dependent compiler frontends and libc implementations) than a language like Python [20]. In-browser IDEs' cross-platform nature also enables support for non-tradtional coding devices like tablets, mobile phones, and Chromebooks. This is a key accessibility consideration for massive online courses, where users may come from around the world and not have access to a standard laptop.

Client-side IDEs offer additional advantages on top of other in-browser IDEs. The key gain from a system that doesn't rely on an external server is the cost-savings. By running on the host device's hardware through the virtualization of a web-browser, client-side IDEs achieve the best of both worlds: avoiding dedicated server costs and system configuration bugs. This setup allows platforms to scale their IDE indefinitely with no variable cost for running code. Further benefits of a client-side IDE include offline access (valuable for courses with students of low-bandwidth connections), zero-cost sharing of code-snippets (a key to the community building aspect of online courses), and seamless integration with Javascript (enabling rich, graphical, interactive coding experiences).

1.2 Contribution

There is currently no fully integrated client-side C++ IDE with all the functionality mentioned. Existing solutions only have pieces. In this paper, we present a toolchain that executes arbitrary student C++ code entirely in the browser, with no reliance on a dedicated server and no cost to scale aside from database reads/writes on code change. Specifically, we can:

- Compile, link, and execute C++ written in the browser.
- Gather blocking standard input from the user
- Include arbitrary additional libraries, with support for exceptions

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CS191W Winter '25, Palo Alto, CA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

- Provide support for graphics through the HTML Canvas API
- Debug student code with line-by-line highlighting, breakpoints, and stepthrough execution
- Execute without dependence on an external server

.

2 Existing Work

There are several prominent in-browser C++ IDEs and two limited demos of client-side C++ IDEs. In the section, we will focus on a selected group of in-browser and client-side tools that represent the state of the art in browser-based C++ IDEs. Other platforms exist, but they represent the same feature sets already displayed, making their inclusion redundant. We'll first examine in-browser IDEs, then understand the WebAssembly ecosystem, then explore client-side IDEs made possible by WASM.

2.1 In-Browser IDEs

2.1.1 Online Python Tutor: C++ [18]. Despite it's name, the Online Python Tutor platform also offers an environment to run and debug C++ code. It's key offerings are a step-by-step debugger (without breakpoints) and a visualization of the program's memory layout. Online Python Tutor's platform works by sending an HTTP request to an external server, running the student's code as well as Valgrind [24] and GDB [7] to collect information about each debugging step, then sending this information back to the Online Python Tutor frontend for display to the user. Notably, this platform is unable to support user input (via `cin, scanf, fgets`, etc.), exceptions, multiple files, the inclusion of non-standard libraries, or graphics.

2.1.2 Online GDB [17]. OnlineGDB is another popular platform for using C++ on the web. It offers an environment to write multi-file C++ projects, run them, collect synchronous user input, handle exceptions, and even use the full-capacity of GDB via its web interface (as the platform's name would suggest). OnlineGDB does this through opening a web socket connection to an external server, and forwarding server terminal output to the frontend and frontend terminal input back to the server. This simulates the experience of the interacting directly with the backend server. Notably, OnlineGDB does not offer any ability to use graphics and requires an external server.

2.2 The WebAssembly Ecosystem

WebAssembly [15] is a portable bytecode format designed as a cross-platform compilation target for languages like C, C++, Rust, Go, etc. Since it is designed to run on any platform, WASM must be executed by a *runtime*. This runtime may be implemented differently depending on the host device, but provides an identical virtual environment for the WASM to execute within on any device. So, through the presence of a WASM runtime capable of running in the browser, any code compiled to WASM can run client-side in the browser.

Low-level languages like C/C++ rely on *syscalls* to interact with the host device to do things (ex. read/write files). Since these syscalls interact directly with the Operating System, they differ depending on whether the host device is running Windows/MacOS/Unix. However, since WASM is cross-platform, there is not a standard set of syscalls for C/C++ programs compiled to WASM to rely on. WASI [28] is a POSIX-like syscall specification that aims to create a shared standard for WASM code interacting with any host OS. Since WASM executes in a virtual runtime, it is up to the runtime to provide *implementations* of these WASI syscalls compatible with the host OS. For example, a WASM runtime on a Unix device would use the Unix syscalls to implement the WASI specification, while a WASM runtime on a Windows device would use the Windows syscalls to provide the functionality of the WASI specification.

2.3 Client-Side IDEs

2.3.1 Emception [9]. Emscripten [5], was one of the original platforms for compiling C++ to something runnable in the browser, even before the adoption of WebAssembly. While it is a crucial tool for compiling enterprise-grade C/C++ applications to WASM to be able to run in the browser, it is a local tool incapable of running on the web. The Emception project aims to remedy this by using Emscripten to compile Emscripten to WebAssembly. The GitHub repo hosts a proof-of-concept that indeed compiles and runs C++ code without any reliance on an external server. However, the platform does not support user input (via `cin, scanf, fgets`, etc.), multiple files, graphics, or debugging.

2.3.2 Binji's WASM Clang [2]. This tool is the most usable client-side C++ tool thus far. Clang [11] recently added support for compiling C++ to WebAssembly using `wasi-sdk`, a customized `libc` that follows the WASI Specification [27]. However, like Emscripten, Clang is a local tool that cannot run in the browser. Binji's WASM Clang was an experimental project for CPPCon 2019 [3] that compiled Clang itself to WebAssembly so it could run in the browser. There were several challenges in that process. Some of Clang's source involved functionality not yet supported in WASM (threads, forking processes) and since the resulting WASM code relied on the WASI specification, browser-specific implementations of WASI syscalls had to be provided. This involved implementing an in-memory virtual filesystem. In the end, the demo hosted in the GitHub repo does in fact compile C++ to WebAssembly in the browser, and even has support for graphics. However, the system cannot handle user input and has no support for debugging.

Although individual pieces of the puzzle exist, there is no system with all the functionality offered by the platform introduced in this paper.

3 System Implementation

In this section, we will describe how each aspect of the system we introduce was implemented. Our toolchain was built atop the IDE featured in Code in Place, which offers a text editor supporting multi-file projects, a terminal to show output and collect input (if the program execution is waiting for it), a panel to display graphics via the HTML Canvas, and a run/stop button. The fully implemented system is displayed in Figures 1 and 2.

3.1 C++ Compilation, Linking, and Execution

3.1.1 Compilation and Linking. To compile, link, and execute student C++ code fully in the browser, we build atop Binji's WASM Clang project mentioned in Section 2. The project provides `Clang`

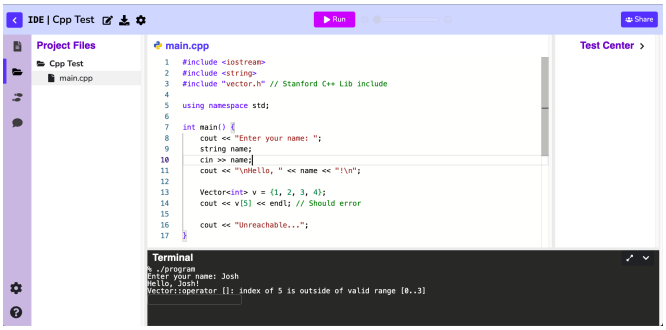

Figure 1: A screenshot of the C++ IDE within Code in Place. Synchronous input, Stanford C++ library, and error handling shown.

Figure 2: A screenshot of the C++ IDE within Code in Place. Graphics capability shown.

(compiler) and `lld` (linker) binaries in WebAssembly that can be run in-browser like any other WASM binary via the Javascript WebAssembly API [13]. Since the WASM code relies on the WASI specification, we needed to provide implementations for these syscalls. However, since we are running the code in browser, there is no base OS syscall library to use from (i.e. the web browser itself has no notion of a filesystem like a Windows/MacOS/Unix operation system does). So, we must provide functionality that simulates these WASI syscalls. To do so, we leveraged Runno [1], which provides an in-browser emulation of each WASI syscall in native Javascript. This is essentially a more-extended version of the in-memory filesystem functionality in Binji's WASM Clang that extends to all of the WASI Specification syscalls.

3.1.2 Execution with Standard Input. The preceding steps turn student C++ code into a WASM binary runnable in the browser via the same Javascript WASM API. However, doing only this would provide a level of functionality no different than Binji's WASM Clang project. To go above and beyond, we first enabled the system to receive standard input from the user.

To do so, we leveraged WebWorkers [14], which execute code independently from the browser's main thread (which renders the UI/processes user interaction, etc). Communication between a Worker instance and the main thread is done via message passing. Additionally, WebWorkers expose the `Atomic.wait()` API which pauses execution until a particular byte of memory in a SharedArrayBuffer is set to an indicated value.

We run the WASM module produced from student C++ code inside a WebWorker. When the binary reaches a call for standard input, it invokes a WASI syscall (`read_fd(1)`) that will then defer to its Javascript implementation. There, the worker sends a message back to the main thread asking for user input and calls `Atomic.wait()` to pause it's execution. Upon receiving this message, the main thread leverages our existing terminal infrastructure to collect input and sets a specific byte of the SharedArrayBuffer. The WebWorker then observes this change, reads a different SharedArrayBuffer containing the input, and continues execution.

3.2 Inclusion of Additional Libraries

Another of our system's contributions is the ability to include arbitrary additional libraries. As a proof-of-concept we choose to include the Stanford C++ library [6] in our compilation process. The library consists of C++ source files and Header files split across several directories. Since our system leverages WASM binaries for `Clang` and `lld`, that are just their source compiled to WebAssembly, including additional code works just as it does for the local versions of these tools. Specifically, we:

(1) Supply the paths of the headers to each compilation step via the `-I` flag.
(2) Run `Clang` on the student's `main.cpp` to produce `main.o` as normal.
(3) For each library source file, run `Clang` on `source.cpp` to produce `source.o`
(4) Run the `lld` linker, supplying `main.o` as well as each of the `source.o` files, to combine them all into one binary.

Our IDE's multi-file project capability and option to upload files enables students/teacher to upload pre-existing library header and source code to be compiled along with their project. Using this system, we've made the Stanford C++ library available for Code in Place students.

Additionally, the Stanford C++ library contains a custom `Error` class that its containers use to signal invalid behavior. WebAssembly does not natively support exceptions, but by modifying the Stanford library error function to use a WASI syscall, we enabled the Stanford C++ code to have readable exceptions which end execution.

3.3 Graphics Support

Another key milestone for our system, and differentiation from other C++ in the browser tools, is the ability to render graphics. There are several popular options for rendering graphics on the web, including the HTML Canvas API [12], WebGL [29], and Three.js [23]. Code in Place already leverages the HTML Canvas API for the graphics and animations found in its assignments, so the natural choice was to integrate our C++ code with this API.

At a high level, we needed to "bind" functions in student C++ code to functions in our existing graphics API. The existing interface between C++ and Javascript code was handled through WASI:

`Clang`'s customized libc was written to use WASI syscalls and we provide implementations for them in Javascript. What we needed to do was add additional syscalls.

To do so, we defined a new sycscall `__wasi_cip_graphics()` in the `wasi/core.h` header file. We then reworked the student-facing APIs of the Stanford C++ library's graphics module to call this syscall with a context object specifying which kind of graphics functionality is expected.

The second half of enabling graphics functionality involved the syscall's Javascript implementation. Such an implementation is supplied by "importing" the JS function into the WASM module, as is done with the WASI implementations provided by Runno. However, the WASM module executes inside a WebWorker, while graphics are rendered by the browser's main thread. So, our graphics syscall's implementation must generate a message to pass from the WebWorker to the host. This message contains context defining what kind of graphics functionality to display. When the main thread receives this message, it decides which of our HTML Canvas graphics functions to invoke.

3.4 Debugging

The final milestone for our client-side C++ IDE was a debugger that enabled line-by-line highlighting, breakpoints, and stepthrough execution.

3.4.1 DWARF Debug Info. While the student is writing C++ code, the browser is actually executing WebAssembly code. This makes displaying state about the program's execution relative to the original C++ code challenging. DWARF files [4] help address this. This format is a compilation stage output that, in addition to containing the normal compiled code, contains debugging information. After changing the `Clang` compile flags to include `-debug-info-kind =standalone, -dwarf-version=4` we get a DWARF containing the following important sections:

(1) `.debug_info` - the core DWARF section, containing nested Debug Information Entries (DIEs) representing functions, variables, types, etc.
(2) `.debug_line` - a "source map" pairing (Web) assembly addresses to C++ source lines.
(3) `.debug_loc` - a table storing original C++ variable locations at particular (Web) assembly instructions.

Each of these sections is crucial to reconstructing the program's execution state relative to the original C++ file.

3.4.2 Controlling WASM Execution. To make use of any of the DWARF's debugging information, we need to snoop the currently executing WASM instruction and invoke a Javascript callback to update the state we track (variable values, function call stack, etc.). Unfortunately, the native Javascript WASM API offers no such functionality. There is no way to inspect the executing instruction, current offset, stack state, etc. of a WASM module. Additionally, leveraging the existing syscall framework to run a JS callback each instruction would require manually inserting a function invocation in between every instruction of the compiled student code binary. This is technically inelegant, would balloon the size of the binary, and would invalidate the critical debug information contained in the DWARF.

Because of these factors, we opted to use WebAssemblyJS, [22] an open source WebAssembly runtime written in pure Javascript, for debugging. It bears the same functionality as the native WASM runtime bundled with Javascript, but since its open source, we could hack it to our needs. Specifically, in the section of code that executes a new instruction, we inserted our own callback function. This allowed us to track relevant debugging information (variable value, function call stack, stack values) as the program executes. Additionally, this level of control allowed us to only execute an instruction when the user requests to, enabling step-by-step execution.

3.4.3 The Complete Debugging Pipeline. We put these two pieces together to implement our complete debugging pipeline. On each instruction execution (whether autonomous program execution or triggered by the student in step-through mode), we:

(1) Compute the address of the currently executing instruction.
(2) Map this address to a line of the student's C++ source file using the `.debug_line` section of the DWARF. Highlight the line.
(3) Check if the value/memory address this instruction is updating is the location of a variable in-scope, as listed by the `.debug_info` section. If so, update its value in our debugging harness and UI.
(4) If we've jumped to a new block that represents a student's function, as listed by the `.debug_info` section, update the call stack in our debugging harness and UI.

Through the above 4 components, we implemented an in-browser, fully client-side C++ IDE that fulfills all the criteria listed in Section 1.2.

4 Tool Evaluation

In this section, we compare our introduced IDE to the existing options explored in Section 2. We compare our tool along two axes: features offered, and compilation + execution time.

4.1 Features Offered

Comparing the suite of features our tool offers to existing in-the-browser C++ options is critical, as we claim to introduce the first tool with all the functionality outlined in Section 1.2.

4.1.1 Blocking standard input. This is a cornerstone of the user experience of an in-browser IDE. Without it, users are forced to either determine their input upfront or cannot provide input at all. This makes for an unintuitive experience, especially in an educational context where learning the interface of a terminal is crucial. Aside from our implementation, only OnlineGDB offers the ability to process user input (both through a static file interface and a terminal).

4.1.2 Include arbitrary additional libraries. The importance of this functionality is two-fold. To be able to replicate code they could write locally, users must be able to access the libraries they might be using in their code. Additionally, most educational platforms make use of a specific library to abstract away some less beginner-friendly aspects of the language. Besides our platform, only OnlineGDB supports this functionality by virtue of their multifile projects.

Feature	Client-Side			In-Browser	
	Our Tool	Binji's Clang	Emception	OnlineGDB	Python Tutor
No External Server	✓	✓	✓		
Blocking Input	✓			✓	✓
Additional Libraries	✓			✓	
Graphics	✓	✓			
Debugger	✓			✓	✓

Table 1: Feature Comparison of Online C++ Execution Tools. A ✓ indicates support for the feature. The double border separates client-side from in-browser tools.

4.1.3 Graphics library. This component is key both to replicating the breadth of code possible locally and, particularly in CS1 contexts, enabling an engaging curriculum/problem set for students. Our platform connects to the HTML Canvas API through a custom syscall, wrapped by a modified version of the Stanford C++ Library. Of the other tools, only Binji's WASM Clang supports a similar functionality, through the WebGL API.

4.1.4 Debugging. Through injecting a custom callback into an open source JS WebAssembly runtime, we track debug state throughout execution and allow students to step through their code. Of the other platforms, Online Python Tutor's C++ environment and OnlineGDB both support similar functionality. OnlineGDB offers a UI over the standard GDB running on the external server. Meanwhile, Online Python Tutor offers step through debugging with line-highlighting and even renders visualizations of the memory state of the user's program. This is an interesting pedagogical tool that could certainly be implemented on our platform.

4.1.5 Execution without a server. Through the use of WebAssembly, the IDE we introduce has no dependency on an external server for any of the functionality it offers. While OnlineGDB and Onlien Python Tutor are both the most full featured in-browser C++ solutions, this is a product of their reliance on a server to run the code, GDB, and Valgrind. Meanwhile, Emception and Binji's WASM Clang offer serverless functionality similar to our platform. However, the are the most bare-bones and are self-described "demoware".

In summary, our tool achieves or exceeds parity with Online Python Tutor and OnlineGDB while greatly exceeding the capabilities of Emception and Binji's WASM Clang. In this way, the platform we introduce is the best of both worlds: the rich functionality of a server-based C++ environment with the no-cost scale of a serverless tool. A visual feature breakdown is included in Table 1.

4.2 Benchmarking

Speed is another critical part of the user experience. When learning to code for this first time, delays to seeing output are frustrating at best and can make students think there is an error at worst. Here, we compare the total time taken to compile, execute, and run code across on each of the platforms discussed and our tool. We tested four programs:

(1) Empty: A minimal program with only an empty main function
(2) Hello: A simple Hello World program
(3) FibComp: A program which computes the 80th Fibonacci number via template metaprogramming, meant to have a long compile time
(4) FibRec: A program which computes the 43rd Fibonacci number via recursion, meant to have a long runtime

Since some platforms have a pre-determined optimization level, we ran both levels `-O0` and `-O2` on the platforms that allow configuration for a more robust comparison. Note that all tests were run on a 2021 Apple M1 Pro 16GB running Google Chrome. The results are visible in Table 2

4.2.1 Online Python Tutor (C++). While Online Python Tutor's platform is a bit faster, our tool remains fairly comparable. It is expected that OPT sees faster execution as it is a server-based platform. However, our tool differed from OPT by at most 1.34s on our benchmarks. Note that we exclude the FibRec benchmark becuase Online Python Tutor imposes a maximum instruction limit (to avoid overloading their visualization framework) that prevented the program from fully executing.

4.2.2 OnlineGDB. As the most robust online C++ tool, it is unsurprising that OnlineGDB is faster than our platform. On our benchmark, Online GDB outperformed our tool by a factor of between 3 and 5 likely due to its use of WebSockets for rapid client to server communication.

4.2.3 Emception. Curiously, Emception's `-O0` times were 2-3x faster than its `-O2` times. This suggests that the compile time is a larger contributer to performance than execution time. A possible explanation for this is that Emception runs other tools, such as Binaryen [26], to optimize its code in the WebAssembly stage. When comparing the `-O2` levels, our tool performs anywhere from 1.2-3.4x better. Across the `-O0` levels, however, our platform and Emception are roughly similar, differing by no more than 1.32 seconds.

4.2.4 Binji Clang. Binji's WASM Clang demo outperformed our tool, by a factor of between 1.5-9.75x. This makes some sense as the demo runs essentially the same core compiler as our tool but forgoes the extra harnesses that integrate our tool into the Code in Place IDE. Additionally, Binji's demo unzips custom WASI libc `.tar` file in the background, while our platform unzips it when compilation is requested. This is another potential area for our tool to make up time.

Platform	**Our Tool**		**Python Tutor**	**OnlineGDB**	**Emception**		**Binji Clang**
Program/Optimization Level	-O0	-O2	-O0	-O0	-O2	-O0	-O2
Empty	2.57s	3.06s	1.72s	0.244s	3.67s	1.74s	0.314s
Hello	3.43s	3.38s	2.29s	0.609s	11.49s	3.21s	0.805s
FibComp	3.39s	3.36s	2.08s	0.451s	10.48s	3.46s	0.95s
FibRec	11.27s	8.66s	8.77s*	4.12s	16.51s	7.09s	5.73s

Table 2: Averaged benchmark results across different platforms and optimization levels. *FibRec for Python Tutor was limited by instruction count at 188.

Program	**Median Performer**	**Median Time (s)**	**Best Time (Our Tool)**	**Abs. Diff (s)**	**Relative Diff (%)**
Empty	Python Tutor (-O0)	1.72	2.57	+0.85	+49.4%
Hello	Python Tutor (-O0)	2.29	3.38	+1.09	+47.6%
FibComp	Python Tutor (-O0)	2.08	3.36	+1.28	+61.5%
FibRec	Emception (-O0)	7.09	8.66	+1.57	+22.1%

Table 3: Comparison of Our Tool's Best Execution Time Against the Median Performer

While our platform does not offer explicit performance gains over the other platforms examined (aside from Emception's `-O2` stage which runs extra optimization steps), our goal was to offer an expanded featureset without sacrificing excessively on performance. Table 3 shows that our platform performs within 1.57s of the median time on each program. While there is room to improve and optimize our tool, 1.5s is a reasonable amount of time to trade for a fully-featured C++ IDE.

5 Limitations and Future Work

While our tool both provides a suite of features on-par with server-based IDEs while being serverless, there are certainly limitations. There are several C++ features that lack built-in WASM support for which our tool does not have a workaround. The only one that is likely to come up in an educational context is Threads. Workarounds may be possible through simulating threads via WebWorkers, but replicating the behavior of Unix threads is quite unlikely.

Another drawback of our tool is it's size. When combined, the `clang` and `lld` binaries are roughly 40 MB. While this doesn't pose a problem on a modern computer and modern browser, this could affect students with low-bandwith connections and older devices. Options for smaller compilers are limited, as few support WebAssembly as a target. Other solutions involve compressing the binaries into Zip files, lazy loading, and browser caching. While these don't change the size of the executable that must be run in the end, they help the page load faster by reducing the amount of bytes loaded initially or not loading the binary at all until necessary.

The considerations around reducing size may also improve execution time, another clear improvement area. While the tradeoff between a second of execution and a full-suite of features makes sense, there are clear areas to improve performance: conditional compilation flags, compiling student code in the background, etc. The goal for our tool should be to get near the performance of Binji's WASM Clang as it is running the same compiler.

The use of that compiler is the source of another limitation. The `clang` and `lld` binaries released by Binji's WASM Clang project are from 2019. They reflect Clang version 8.0.1, which supports C++17 and partially C++20. Currently, Clang is in version 20 which supports C++23. Most of the differences here are transparent to a CS1 student, however, future work would use the most recent Clang to allow for maximum flexibility.

In addition to the items mentioned above, future work takes two main forms. First, improving debugging capabilities is a priority, as it is one of the most fundamental skills for new programmers. Future iterations of the tool will see memory diagrams similar to those of Online Python Tutor. The other primary direction is to extend to more languages. WebAssembly is a portable target that countless languages can be compiled to. Most of the tooling we introduced (connection to HTML Canvas, synchronous input, debugging, etc.) lives on the half of the stack after WASM. What this means is that much of the existing infrastructure will function just the same with other languages (ex. Rust, Go). As additional languages support compilation to WASM, those compilers can be compiled to WASM and run on the web.

Most excitingly, our IDE will be hosted live during the next iteration of Code in Place. This will be an outstanding opportunity to stress test the system with thousands of real students and gather data on utility, user experience, and learning outcomes with the tool.

Acknowledgments

This project would not be possible without the mentorship of Professor Chris Piech, and his visionary platform Code in Place. Code in Place is generously supported by the Carina Foundation and a team of talented developers. A particular thanks to TJ Jefferson for outstanding work Code in Place's original client-side Python IDE. It heavily inspired this project.

References

[1] Ben Taylor. 2024. Runno: Browser-Based Code Execution Environment. https://github.com/taybenlor/runno/tree/main Accessed: March 19, 2025.

[2] binji. 2024. Wasm-Clang: Clang Toolchain for WebAssembly. https://github.com/binji/wasm-clang Accessed: March 19, 2025.

[3] CppCon. 2024. CppCon: C++ Conference Official Website. http://CppCon.org/ Accessed: March 19, 2025.

[4] DWARF Debugging Standard Committee. 2024. DWARF Debugging Information Format Standard. https://dwarfstd.org/ Accessed: March 19, 2025.

[5] Emscripten. 2024. Emscripten: WebAssembly Compiler Toolchain. https://emscripten.org/index.html Accessed: March 19, 2025.

[6] Eric S. Roberts and Julie Zelenski. 2024. Stanford C++ Library. https://github.com/zelenski/stanford-cpp-library/tree/master Accessed: March 19, 2025.

[7] GNU Project. 2024. GDB: The GNU Project Debugger. https://sourceware.org/gdb/ Accessed: March 19, 2025.

[8] Thomas Jefferson, Chris Gregg, and Chris Piech. 2024. PyodideU: Unlocking Python Entirely in a Browser for CS1. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 583–589.

[9] Jose Prendes. 2024. Emception: WebAssembly-based C++ Emulator. https://github.com/jprendes/emception Accessed: March 19, 2025.

[10] Johannes Krugel and Peter Hubwieser. 2020. Web-based learning in computer science: Insights into progress and problems of learners in MOOCs. *Non-Formal and Informal Science Learning in the ICT Era* (2020), 51–79.

[11] LLVM Project. 2024. Clang: C Language Family Frontend for LLVM. https://clang.llvm.org/ Accessed: March 19, 2025.

[12] MDN Web Docs. 2024. Canvas API. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API Accessed: March 19, 2025.

[13] MDN Web Docs. 2024. Using the WebAssembly JavaScript API. https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Using_the_JavaScript_API Accessed: March 19, 2025.

[14] MDN Web Docs. 2024. Using Web Workers API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers Accessed: March 19, 2025.

[15] MDN Web Docs. 2024. WebAssembly Documentation. https://developer.mozilla.org/en-US/docs/WebAssembly Accessed: March 19, 2025.

[16] MDN Web Docs. 2024. WebSockets API. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API Accessed: March 19, 2025.

[17] OnlineGDB. 2024. OnlineGDB: Online C++ Compiler. https://www.onlinegdb.com/online_c++_compiler Accessed: March 19, 2025.

[18] Philip Guo. 2024. Python Tutor: C++ Visualizer. https://pythontutor.com/cpp.html#mode=edit Accessed: March 19, 2025.

[19] Christopher Piech, Ali Malik, Kylie Jue, and Mehran Sahami. 2021. Code in place: Online section leading for scalable human-centered learning. In *Proceedings of the 52nd acm technical symposium on computer science education*. 973–979.

[20] Larissa Salerno, Christoph Treude, and Patanamon Thongtatunam. 2024. Open Source Software Development Tool Installation: Challenges and Strategies For Novice Developers. arXiv:2404.14637 [cs.SE] https://arxiv.org/abs/2404.14637

[21] Mark Stehlik, Erin Cawley, and David Kosbie. 2020. Cmu cs academy: A browser-based, text-based introduction to programming through graphics and animations in python. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*. 1420–1420.

[22] Sven Sauleau and Contributors. 2024. WebAssembly.js: WebAssembly Toolkit for JavaScript. https://github.com/xtuc/webassemblyjs Accessed: March 19, 2025.

[23] Three.js Project. 2024. Three.js: JavaScript 3D Library. https://threejs.org/ Accessed: March 19, 2025.

[24] Valgrind Developers. 2024. Valgrind: Memory Debugging, Profiling, and Leak Detection Tool. https://valgrind.org/ Accessed: March 19, 2025.

[25] WebAssembly. 2024. WebAssembly Official Website. https://webassembly.org/ Accessed: March 19, 2025.

[26] WebAssembly Community Group. 2024. Binaryen: Compiler and Toolchain Infrastructure for WebAssembly. https://github.com/WebAssembly/binaryen Accessed: March 19, 2025.

[27] WebAssembly Community Group. 2024. WASI-SDK: WebAssembly System Interface SDK. https://github.com/WebAssembly/wasi-sdk Accessed: March 19, 2025.

[28] WebAssembly System Interface (WASI). 2024. WASI: WebAssembly System Interface. https://wasi.dev/ Accessed: March 19, 2025.

[29] WebGL Project. 2024. WebGL: 3D Graphics for the Web. https://get.webgl.org/ Accessed: March 19, 2025.